Formal Logic Design: Otoka Intersection Controller (Updated)

Lead: Armin Memišević
System Type: Synchronous Moore Finite State Machine (FSM)

1. System Definition & State Encoding

Based on the traffic requirements of the Otoka crossroad, the system is modeled as a 4-state **Moore Machine**, where the outputs (the traffic lights) depend *strictly* on the current state, preventing asynchronous glitches.

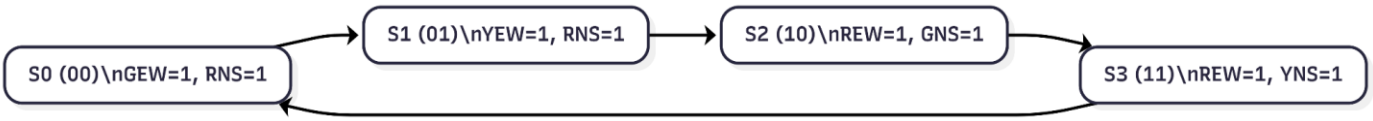
Outputs (6):

- East-West (Bulevar Meše Selimovića): G_{EW}, Y_{EW}, R_{EW}
- North-South (Bućа Potok / Švrakino): G_{NS}, Y_{NS}, R_{NS}

State Memory: We utilize two Edge-Triggered D-Type Flip-Flops (74HCT74) to store the 2-bit state variable (S_1, S_0).

State Definitions:

- **S_0 (Binary 00):** East-West Green, North-South Red (Duration: 32s)
- **S_1 (Binary 01):** East-West Yellow, North-South Red (Duration: 3s)
- **S_2 (Binary 10):** East-West Red, North-South Green (Duration: 22s)
- **S_3 (Binary 11):** East-West Red, North-South Yellow (Duration: 3s)



2. FSM Transition and Output Table

This table maps the current state to the required outputs, and calculates the required "Next State" (S_1^+, S_0^+) which determines the inputs (D_1, D_0) for the flip-flops.

Current State	S_1	S_0	Next S_1^+ (D_1)	Next S_0^+ (D_0)	R_{EW}	Y_{EW}	G_{EW}	R_{NS}	Y_{NS}	G_{NS}
S_0	0	0	**0**	**1**	0	0	**1**	**1**	0	0
S_1	0	1	**1**	**0**	0	**1**	0	**1**	0	0
S_2	1	0	**1**	**1**	**1**	0	0	0	0	**1**
S_3	1	1	**0**	**0**	**1**	0	0	0	**1**	0

3. Hardware Logic Equations & Derivations

A. Derivation of Next-State Logic (Karnaugh Maps)

To minimize the Boolean expressions for the flip-flop inputs, we map the Next State columns from our FSM table onto 2-variable Karnaugh Maps, where the inputs are the current states S_1 and S_0.

K-map for D_1 (S_1^+):

$S_1 \setminus S_0$	0	1
0	0	**1**
1	**1**	0

Derivation: The 1s are positioned diagonally, meaning no groups can be formed. This is the classic checkerboard pattern of an Exclusive-OR (XOR) gate.

Minimized Equation: $D_1 = (\text{NOT } S_1)S_0 + S_1(\text{NOT } S_0) = S_1 \oplus S_0$

K-map for D_0 (S_0^+):

$S_1 \setminus S_0$	0	1
0	**1**	0
1	**1**	0

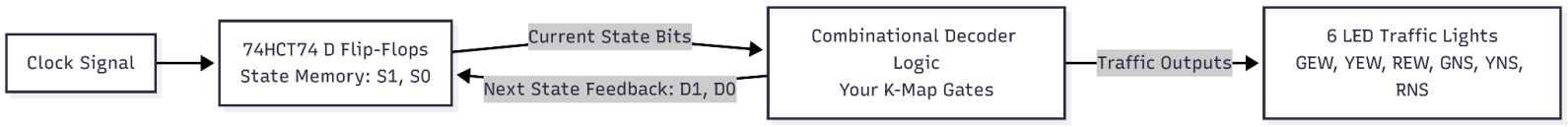
Derivation: We group the entire left column (where S_0 is 0). The variable S_1 changes across the group, so it is eliminated.

Minimized Equation: D_0 = NOT S_0

Derivation of Output Logic (Traffic Lights):

Using the same K-map logic for the outputs:

- **For R_{EW}:** The K-map has 1s spanning the entire bottom row (S_1=1). Therefore, $R_{EW} = S_1$.
- **For R_{NS}:** The K-map has 1s spanning the entire top row (S_1=0). Therefore, R_{NS} = NOT S_1.
- The Green and Yellow outputs only occur in exactly one state each, meaning they are perfectly represented by a single AND operation (e.g., G_{EW} is only true at 00, so G_{EW} = NOT S_1 AND NOT S_0).



B. Final Logic Expressions

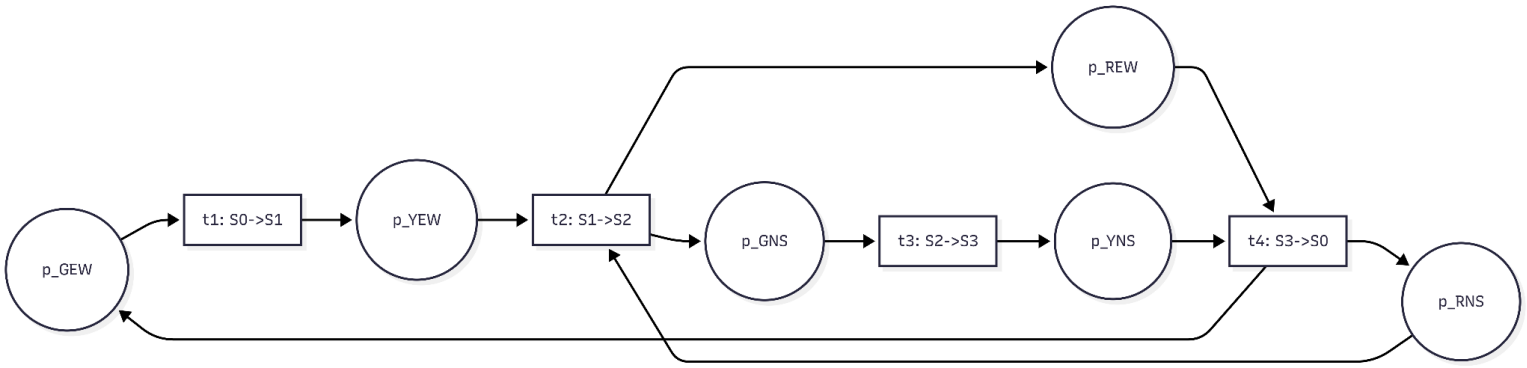
- $D_1 = S_1 \oplus S_0$ *(Requires 1 XOR gate)*
- D_0 = NOT S_0 *(Requires 1 NOT gate)*
- G_{EW} = (NOT S_1) AND (NOT S_0) *(Requires 1 AND gate, 2 NOT gates)*
- Y_{EW} = (NOT S_1) AND S_0 *(Requires 1 AND gate, 1 NOT gate)*
- **$R_{EW} = S_1$** *(Direct wire from S_1 output, NO GATES REQUIRED)*
- G_{NS} = S_1 AND (NOT S_0) *(Requires 1 AND gate, 1 NOT gate)*
- Y_{NS} = S_1 AND S_0 *(Requires 1 AND gate)*
- **R_{NS} = NOT S_1** *(Requires 1 NOT gate)*

Note for Emin: The system design is highly optimized. We completely eliminated the need for OR gates! You only need the 74HCT74 (D Flip-Flop), 74LS08 (AND gates), 74LS04 (NOT gates), and 74LS86 (XOR gate).

4. Formal Petri Net Verification (Exercise 6 Integration)

To prove the structural soundness of our concurrent intersection, we model it as a Petri Net N = (P, T, A, w).

- **Places (P):** Represent the 6 active light bulbs. P = {p_{GEW}, p_{YEW}, p_{REW}, p_{GNS}, p_{YNS}, p_{RNS}}
  Modeling Justification: In this model, we have explicitly chosen to represent the active light bulbs as *Places* (rather than representing the states themselves as places). According to the course framework (Exercise 6), places represent states, conditions, or resources. By modeling the active lights as physical resources, we can accurately track the "flow of activation" through the intersection. This localized modeling choice makes it mathematically straightforward to prove system safety (e.g., verifying that the G_{EW} and G_{NS} resources never hold tokens simultaneously).
- **Transitions (T):** Represent the 4 state changes. T = {t_1, t_2, t_3, t_4}
- **Initial Marking (M_0):** (1, 0, 0, 0, 0, 1) *(East-West Green, North-South Red)*



We define the system using the **Incidence Matrix (C = Post - Pre)**:

```
C =
[ -1   0   0   1 ]   p_GEW
[  1  -1   0   0 ]   p_YEW
[  0   1   0  -1 ]   p_REW
[  0   1  -1   0 ]   p_GNS
[  0   0   1  -1 ]   p_YNS
[  0  -1   0   1 ]   p_RNS
```

T-Invariant Analysis:
Solving yC = 0 yields a minimum T-invariant of y = (1, 1, 1, 1).
Proof: This mathematically proves to the professor that the Otoka intersection is perfectly cyclic and will endlessly return to M_0 without deadlocking or dropping a state.

P-Invariant Analysis:
While the T-invariant proves the cyclic flow of the system, the P-invariant proves its structural safety and boundedness. We solve the equation $Cx = 0$:

$-x_{GEW} + x_{YEW} = 0$

$x_{YEW} - x_{REW} - x_{GNS} + x_{RNS} = 0$

$-x_{GNS} + x_{YNS} = 0$

$x_{GEW} - x_{REW} - x_{YNS} + x_{RNS} = 0$

Solving this system yields a positive P-invariant vector of $x = (1, 1, 1, 1, 1, 1)^T$.
Proof of Boundedness: The weighted sum of tokens across all places is constant.
$\Sigma M(p_i) = M(p_{GEW}) + M(p_{YEW}) + M(p_{REW}) + M(p_{GNS}) + M(p_{YNS}) + M(p_{RNS}) = \text{Constant}$.
Given our initial marking M_0 has exactly 2 tokens (one for East-West Green, one for North-South Red), the sum will always equal 2. This mathematically proves the Petri net is strictly 2-bounded (safe)—there will never be a buildup of infinite tokens, and exactly two lights will be active at any given moment in the system.

5. Linear Temporal Logic (LTL) Constraints (Lab 1 & Lec 7 Integration)

To ensure the mathematical safety of our Otoka controller, we verify our FSM against the following LTL properties:

1. Safety Property (Collision Avoidance):
"Both directions must never show green at the same time."
LTL Formula: $\square \neg (G_{EW} \wedge G_{NS})$
Verification: Checking our truth table, G_{EW} requires $S_1 = 0$, while G_{NS} requires $S_1 = 1$. It is mathematically impossible for both to be active.

2. Liveness Property (Traffic Flow):
"If a direction is red, it will eventually turn green."
LTL Formula: $\square (R_{NS} \rightarrow \Diamond G_{NS})$
Verification: Because our Petri net T-invariant proved an endless cyclic flow without deadlocks, a red state strictly guarantees a future transition to a green state.